Amazon-M2 Next Product Recommendation

Session-Based Sequential Recommendation with Cross-Attention over Item Attributes

Mikey Ferguson

April 29, 2026

The University of Tulsa

Graduate course final project

Outline

Problem

Pipeline

Novel Method

Results

Conclusion

Problem

Task

Goal

Given a shopping session, rank the next product the user will interact with.

Input

- Ordered product history: $[i_1, i_2, \ldots, i_t]$
- Session id used as the user field for RecBole
- Product metadata: title, brand, color, price

Evaluation

- Full-sort ranking over the item vocabulary
- MRR@100, Recall@100, NDCG@100
- Reported overall and per locale

Dataset and task follow Amazon-M2 Task 1 from the KDD Cup 2023 multilingual recommendation challenge.

Dataset Scale

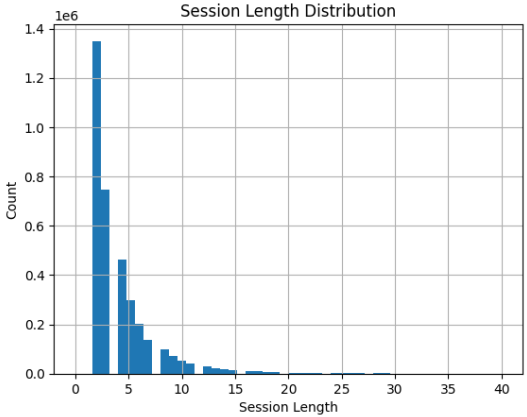
Locale	Train sessions
UK	1.18M
DE	1.11M
JP	979K
IT	127K
FR	118K
ES	89K

- About 3.6M sessions across 1.4M products
- Six locales with a heavy imbalance between large and small markets
- Sessions are short: mean about 4 items, median 3
- Catalog is long-tailed and multilingual

Practical consequence

The model must learn from short sequences while ranking against a very large product vocabulary.

Session Length Distribution



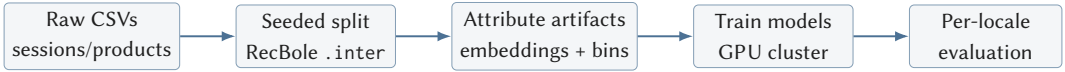
Items per session

Statistic	Value
Count	3,606,249
Mean	4.24
Std. dev.	3.55
Min	2
25%	2
Median	3
75%	5
Max	474

Heavy right skew: most sessions are 2–3 items; the tail stretches into the hundreds.

Pipeline

Implementation Pipeline



Reproducibility

- Seeded 80/10/10 split with `SEED=42`
- Evaluation loads latest checkpoint by model name
- Per-locale test files preserve RecBole schema

Scale handling

- Full-data training runs on the GPU cluster
- Smoke tests use `--nrows`
- Plot scripts parse Slurm JSON logs directly

Baselines

Model	Role	Implementation detail
Popularity	Non-neural baseline	Hand-rolled counter over train histories and targets; RecBole `Pop` was incompatible with the sequential setup.
GRU4Rec	Sequential baseline	RecBole GRU encoder over item-id sequences with cross-entropy ranking loss.
NARM	Attention baseline	RecBole neural attentive recommendation model for session-based ranking.
NovelModel	Proposed method	GRU session encoder plus cross-attention over title, brand, color, and price attributes.

Popularity is reported honestly as a negative baseline on this split; its paper-level performance is not reproduced under this evaluation protocol.

Baseline Instantiation in RecBole

scripts/train.py

```
from recbole.quick_start import run_recbole

run_recbole(
    model='GRU4Rec',   # or 'NARM'
    dataset='amazon_m2',
    config_file_list=['scripts/config.yaml'],
)
```

scripts/config.yaml (key entries)

```
USER_ID_FIELD: session_id
ITEM_ID_FIELD: item_id
benchmark_filename: [train, valid, test]
alias_of_item_id: [item_id_list]
filter_inter_by_user_or_item: False
metrics: [MRR, Recall, NDCG]
topk: [100]
valid_metric: MRR@100
```

GRU4Rec and NARM need no model code — swapping baselines is a one-line change to the `model=` string. Pop is the one exception (`scripts/pop_baseline.py`) because RecBole's built-in `Pop` is incompatible with our sequential config; in hindsight its near-zero numbers match the broken RecBole version exactly, so the custom code turned out to be unnecessary. The novel method, by contrast, is a custom `SequentialRecommender` subclass (next section).

Novel Method

Motivation

Amazon-M2 paper, Table 5 (Task 2 ablation, ES/FR/IT supervised)

The authors initialize the item-embedding table with Multilingual Sentence-BERT *title* vectors (variants SRGNNF, GRU4RecF). MRR@100 and Recall@100 drop on every locale tested. Their conclusion: *"how to effectively utilize textual features remains an open question."*

Variant	MRR	Recall	Variant	MRR	Recall
SRGNN	0.2640	0.5429	GRU4Rec++	0.2564	0.5856
+ title (F)	0.2303	0.4976	+ title (F)	0.2427	0.5454

Our angle — un-naive feature fusion

	Paper (SRGNNF / GRU4RecF)	This work (NovelModel)
Item-id embedding init	MiniLM title vectors	Random (RecBole default)
How attributes enter	Baked into the id-embedding row	Separate per-item bag via cross-attention
Attribute set	Title only	Title, brand, color, price bin

Attribute Construction

Text attributes

- Title, brand, and color
- Encoded with `paraphrase-multilingual-MiniLM-L12-v2`
- 384-dimensional, L2-normalized vectors
- Aligned to RecBole internal item ids

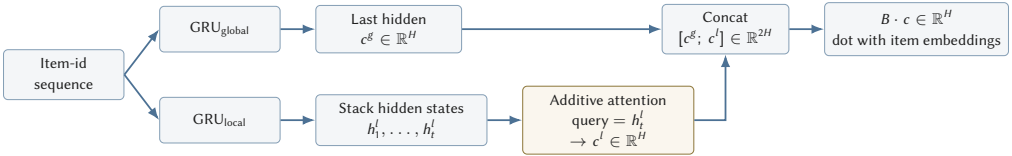
Price

- 32 train-only quantile bins
- Learned embedding per bin
- Robust to extreme catalog price outliers
- Reserved sentinel for padding or missing bins

Catalog coverage

Every item referenced by the session data has a catalog row, so the model does not need an unknown-attribute fallback.

NARM: Two GRUs with Attention (Li et al., 2017)

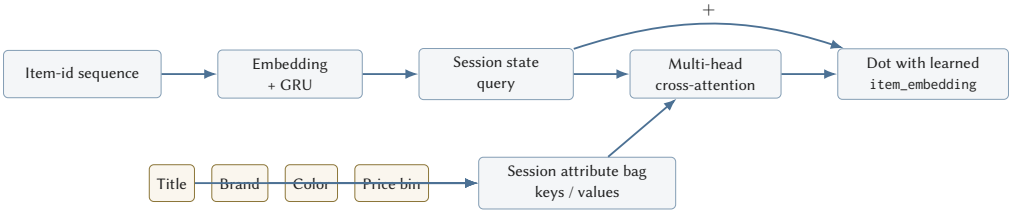


- Two GRUs over the same item embeddings.
- Global side: last hidden state only.
- Local side: stack of hidden states fed to an attention block, query $= h_t^l$.
- Project $[c^g;\ c^l]$ through $B \in \mathbb{R}^{H \times 2H}$; dot with every item embedding.

Attention vs. NovelModel (next slide)

- NARM: $Q = h_t^l$; $K = V$ = stacked GRU states.
- Ours: Q = session state; $K = V$ = stacked attribute bag.
- Same K=V; NARM additive, ours scaled dot-product.

NovelModel Architecture



Shapes used on the next slide: B = batch size, T = padded session length, H = `hidden_size` = 128, K = `len(self.active_item_attributes)` $\in \{1, 2, 3, 4\}$, `n_items` $\approx$ 1.4M.

The score box reuses the *same* `nn.Embedding(n_items, H)` that turns input ids into vectors for the GRU on the left — tied input/output weights.

NovelModel forward walkthrough

```
def forward(self, item_seq, item_seq_len):
    # item_seq: (B, T) item ids, 0 = PAD.  item_seq_len: (B,) real lengths.
    x          = self.emb_dropout(self.item_embedding(item_seq))   # (B, T, H)
    gru_out, _ = self.gru(x)                # (B, T, H); 2nd return (n_layers, B, H) discarded
    state      = self.gather_indexes(gru_out, item_seq_len - 1)    # (B, H)
    K = len(self.active_item_attributes)
    proj = []
    for attr in self.active_item_attributes:                       # iterates 1..4 active slots
        if   attr == 'title': proj.append(self.title_proj(self.title_embeddings))
        elif attr == 'brand': proj.append(self.brand_proj(self.brand_embeddings))
        elif attr == 'color': proj.append(self.color_proj(self.color_embeddings))
        elif attr == 'price': proj.append(self.price_embedding(self.price_bin_idx))
    attr_bag = torch.stack(proj, dim=1)                            # (n_items, K, H)
    sess_bag = attr_bag[item_seq].flatten(1, 2)                    # (B, T*K, H)
    pad_mask = (item_seq == 0).repeat_interleave(K, dim=1)         # (B, T*K)
    attn, _  = self.cross_attn(state.unsqueeze(1), sess_bag, sess_bag,
                               key_padding_mask=pad_mask)          # (B, 1, H)
    return state + attn.squeeze(1)                                 # (B, H)
```

Logits in `calculate_loss`: `output @ self.item_embedding.weight.T`, shape $(B,$ `n_items`$)$. The `item_embedding` on line 3 above and the score node of the diagram are the same tensor.

Training Configuration

Shared setup

- RecBole 1.2.0 with PyTorch
- Full-sort evaluation
- Top-$K = 100$
- Metrics: MRR, Recall, NDCG

NovelModel

- Hidden size: 128
- GRU layers: 1
- Dropout: 0.3
- Attention heads: 4
- Loss: cross entropy over all items

Ablation hook

`attribute_slots` can train title-only, title+brand, title+brand+price, or full title+brand+color+price variants without changing model code.

Results

Evaluation Metrics

Each test session has exactly one held-out next item. Let r_s be the rank of that item in the model's full-vocabulary ranking for session s; a session contributes 0 when $r_s > 100$. All three metrics are means over the N test sessions.

- MRR@100 $= \frac{1}{N}\sum_s \frac{1}{r_s}\mathbf{1}[r_s \leq 100]$. Reciprocal rank — rank 1 scores 1.0, rank 10 scores 0.1, rank 100 scores 0.01. Punishes slipping down the list.
- Recall@100 $= \frac{1}{N}\sum_s \mathbf{1}[r_s \leq 100]$. Equivalent to HitRate@100 here (single positive per session): did the true item make the top 100, yes or no.
- NDCG@100 $= \frac{1}{N}\sum_s \frac{1}{\log_2(r_s+1)}\mathbf{1}[r_s \leq 100]$. Log-discounted hit; same shape as MRR but gentler — rank 1 scores 1.0, rank 10 scores ≈ 0.29, rank 100 scores ≈ 0.15.

Recall is the easiest to satisfy; MRR is the strictest. NDCG sits between them.

Overall Test Results

This work — all six locales

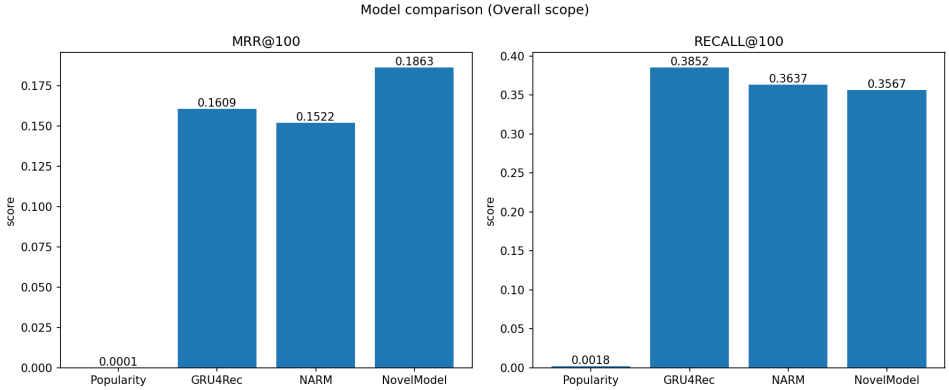
Model	MRR@100	Recall@100	NDCG@100
Popularity	0.0001	0.0018	0.0004
NARM	0.1522	0.3637	0.1951
GRU4Rec	0.1609	**0.3852**	0.2061
NovelModel	**0.1863**	0.3567	**0.2216**

Amazon-M2 paper, Task 1 — UK/DE/JP-only Overall *Table 3 + Appendix C Table 7*

Model	MRR	Recall	NDCG	Model	MRR	Recall	NDCG
Popularity	**0.2426**	0.4243	0.2175	SRGNN	0.1948	0.4032	0.2448
GRU4Rec++	0.1757	0.3808	0.2374	CORE	0.1609	**0.5591**	**0.2839**
NARM	0.1908	0.4180	0.2452	MGS	0.1907	0.5194	0.2471
STAMP	0.2159	0.3512	0.2379				

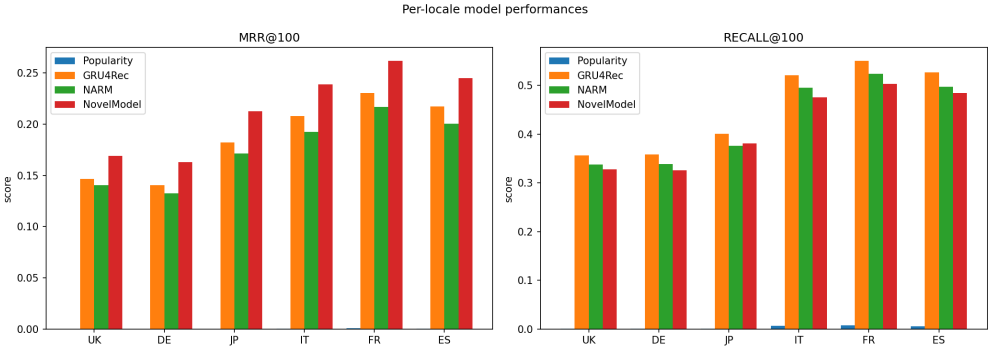
The two "Overall" columns live in different universes: ours unions all six locales, the paper's unions only UK/DE/JP. NovelModel beats both deep baselines on MRR and NDCG; GRU4Rec retains the Recall lead by ∼3 points.

Model Comparison Plot



Two metrics shown (MRR@100 and Recall@100) — the strictest rank-sensitive metric and the loosest hit-rate metric. NDCG@100 sits between them; numerical values appear in the previous table.

Per-Locale Behavior



The smaller locales score higher here; that makes the all-six-locale result a different object from the paper's UK/DE/JP-only overall number.

Conclusion

Conclusion

Deliverables

- Reproducible RecBole pipeline: preprocessing $\rightarrow$ training $\rightarrow$ per-locale evaluation, seeded 80/10/10 split across all six locales.
- GRU4Rec, NARM, and a hand-rolled Popularity baseline; per-locale numbers reach paper-ballpark on the UK/DE/JP weighted row.
- NovelModel: custom `SequentialRecommender` subclass with cross-attention over a title / brand / color / price-bin attribute bag, plus a slot-toggle ablation surface wired through `train.py`.

Limitations

- Our six-locale "Overall" $\neq$ the paper's UK/DE/JP-only "Overall" — different universes; both reported with scope labels.
- GRU4Rec sits $\sim$10–12% behind paper Table 3 GRU4Rec++ on the UK/DE/JP-weighted row — seeded-split + augmentation gap, not a bug.
- NovelModel was trained without a hyperparameter search; numbers reflect a single configuration (`hidden_size=128`, 4 attention heads, full slot list).
- Paper Popularity (MRR 0.2426) is unreproducible: their protocol's definition of "most popular" differs from ours (per-locale vocabulary, leave-one-out test, or the official KDD Cup test set), but the release does not spell out which.

Future Work

Where this goes next

- **Recall-targeted variant.** Swap the GRU for a CORE or MGS encoder under the same attribute-bag fusion — target their higher Recall@100 ($\sim$0.55 vs GRU4Rec++'s 0.38).
- **Slot-toggle ablations.** Title only $\rightarrow$ +brand $\rightarrow$ +color $\rightarrow$ +price. Infrastructure is already wired through `train.py --slots` and `run_training.sbatch`; only the cluster jobs remain. Direct comparison to paper Table 5's title-only failure case.

Questions?